Creating an AR Web Browser in Unity with Voice and Keyboard Input

Overview

This Unity AR Web Browser application will:

1. Render a web browser in AR using a plane or canvas.
2. Accept URL or search queries via:
   - Voice commands (using Unity's Speech-to-Text support or a third-party service like Google Speech-to-Text).
   - A virtual AR keyboard for text input.

Steps

1. Set Up Unity Project

- **Unity Version**: Use Unity 2022.3.56f1 LTS.
- **AR Setup**:
  - Install AR Foundation and the relevant XR plugin (ARCore for Android or ARKit for iOS) via Unity's Package Manager.
- **Add Dependencies**:
  - Install a web browser plugin:
    - **UniWebView** or **Embedded Browser** from the Unity Asset Store.

2. Create AR Scene

1. Add the **AR Session and **AR Camera** prefabs to your scene.**

2. Add a **Plane Detection script to place objects in AR.**

3. Create Web Browser Plane

1. Create a **3D Plane in the scene (to display the browser content).**

2. Attach a **Renderer component to the plane.**

3. Position the plane in the AR environment using AR plane detection.

4. Add AR Keyboard

1. Create a simple UI keyboard using Unity's **Canvas and **Button** components.**

2. Map each key to a script that appends text to an input field.

5. Add Voice Input

1. Use a voice-to-text plugin (e.g., Google Speech-to-Text, Unity's DictationRecognizer).

2. Transcribe voice input into text and load the corresponding URL.

Scripts

Script 1: Browser Controller

```csharp
using UnityEngine;

public class BrowserController : MonoBehaviour
{
    public string defaultUrl = "https://www.google.com";
    public GameObject browserPlane;

    void Start()
    {
        LoadWebPage(defaultUrl);
    }

    public void LoadWebPage(string url)
    {
        Debug.Log($"Loading URL: {url}");
        // Use plugin API to load URL on the plane
    }
}
```

Script 2: AR Keyboard Manager

```csharp
using UnityEngine;
using UnityEngine.UI;

public class ARKeyboardManager : MonoBehaviour
{
    public InputField inputField;
    public BrowserController browserController;

    public void OnKeyPress(string key)
    {
        inputField.text += key;
    }

    public void OnEnter()
    {
        browserController.LoadWebPage(inputField.text);
    }
}
```

Script 3: Voice Input Manager
```csharp
using UnityEngine;
using UnityEngine.Windows.Speech;
```

```
public class VoiceInputManager : MonoBehaviour
{
    private DictationRecognizer dictationRecognizer;
    public BrowserController browserController;

    void Start()
    {
        dictationRecognizer = new DictationRecognizer();
        dictationRecognizer.DictationResult += (text, confidence) =>
        {
            Debug.Log("Voice input: " + text);

browserController.LoadWebPage("https://www.google.com/search?q
=" + text);
        };
        dictationRecognizer.Start();
    }

    void OnDestroy()
    {
        if (dictationRecognizer != null)
        {
            dictationRecognizer.Stop();
            dictationRecognizer.Dispose();
        }
    }
```

}
```

6. Build and Deploy

- Build for Android (ARCore) or iOS (ARKit).

- Test the application on a physical device.
